Exercise 1: Dataset Understanding and Preprocessing

1. Load the dataset and display .head(), .info(), and .describe().

In [ ]:
```
import pandas as pd
import os

import kagglehub
path = kagglehub.dataset_download("awaiskaggler/insurance-csv")

csv_files = [f for f in os.listdir(path) if f.endswith('.csv')]

if csv_files:
    csv_file_path = os.path.join(path, csv_files[0])
    df = pd.read_csv(csv_file_path)
    print("Dataset loaded successfully.")
else:
    print("No CSV files found in the downloaded directory.")
```

```
Using Colab cache for faster access to the 'insurance-csv' dataset.
Dataset loaded successfully.
```

In [ ]:
```
# Display first 5 rows of the data
display(df.head())
```

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86

In [ ]:
```
# Display the summary of the data
display(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   expenses  1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
```

In [ ]:
```
# Display the desccriptive statistics of the dataframe.
df.describe()
```

Out[ ]:

	age	bmi	children	expenses
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.665471	1.094918	13270.422414
std	14.049960	6.098382	1.205493	12110.011240
min	18.000000	16.000000	0.000000	1121.870000
25%	27.000000	26.300000	0.000000	4740.287500
50%	39.000000	30.400000	1.000000	9382.030000
75%	51.000000	34.700000	2.000000	16639.915000
max	64.000000	53.100000	5.000000	63770.430000

In [ ]:
```
df.describe(include='object')
```

Out[ ]:

	sex	smoker	region
count	1338	1338	1338
unique	2	2	4
top	male	no	southeast
freq	676	1064	364

2. Identify categorical and numerical columns

In [ ]:
```
cat_cols = df.select_dtypes(include='object').columns
num_cols = df.select_dtypes(include=['int64', 'float64']).columns

print(f"Categorical columns: {cat_cols}")
print(f"Numerical columns: {num_cols}")
```

```
Categorical columns: Index(['sex', 'smoker', 'region'], dtype='object')
Numerical columns: Index(['age', 'bmi', 'children', 'expenses'], dtype='object')
```

3. Check for the missing values if any and handle them

In [ ]:
```
df.isnull().any()
```

Out[ ]:

	0
age	False
sex	False
bmi	False
children	False
smoker	False
region	False
expenses	False

dtype: bool

4. Encode categorical features using:

- Label Encoding
- One-Hot Encoding

In [ ]:

```
# Applying label encoding to 'sex and smoker'.
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['sex_encoded'] = label_encoder.fit_transform(df['sex'])
df['smoker_encoded'] = label_encoder.fit_transform(df['smoker'])
df.head()
```

Out[ ]:

	age	sex	bmi	children	smoker	region	expenses	sex_encoded	smoker_encoded
0	19	female	27.9	0	yes	southwest	16884.92	0	1
1	18	male	33.8	1	no	southeast	1725.55	1	0
2	28	male	33.0	3	no	southeast	4449.46	1	0
3	33	male	22.7	0	no	northwest	21984.47	1	0
4	32	male	28.9	0	no	northwest	3866.86	1	0

In [ ]:

```
# Applying one-hot encoding to 'region' column
df = pd.get_dummies(df, columns=['region'], prefix=['region'])
df.head()
```

Out[ ]:

	age	sex	bmi	children	smoker	expenses	sex_encoded	smoker_encoded	region_northeast
0	19	female	27.9	0	yes	16884.92	0	1	False
1	18	male	33.8	1	no	1725.55	1	0	False
2	28	male	33.0	3	no	4449.46	1	0	False
3	33	male	22.7	0	no	21984.47	1	0	False
4	32	male	28.9	0	no	3866.86	1	0	False

5. Normalize/standardize numerical features.

In [ ]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
for col in num_cols:
  df[col] = scaler.fit_transform(df[[col]])
df.head()
```

Out[ ]:

	age	sex	bmi	children	smoker	expenses	sex_encoded	smoker_encoded	regi
0	-1.438764	female	-0.453646	-0.908614	yes	0.298583	0	1	
1	-1.509965	male	0.514186	-0.078767	no	-0.953689	1	0	
2	-0.797954	male	0.382954	1.580926	no	-0.728675	1	0	
3	-0.441948	male	-1.306650	-0.908614	no	0.719843	1	0	
4	-0.513149	male	-0.289606	-0.908614	no	-0.776802	1	0	

6. Split the dataset into training and testing sets (80:20).

In [ ]:

Exercise 2: Apply Machine Learning Regression Models

Apply the following regression models to predict charges:

- Simple Linear Regression (using bmi only)
- Multiple Linear Regression (all features)
- Polynomial Regression (degree 2 and 3)
- Ridge Regression (with tuning alpha)
- Lasso Regression (with tuning alpha)
- Decision Tree Regressor

For each model

- Train the model
- Predict on test data
- Evaluate and record:
    1. MAE
    2. MSE
    3. RMSE
    4. R^2 Score
- Create scatter plot of Actual vs Predicted values

In [ ]:
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, root
from sklearn.preprocessing import PolynomialFeatures


X = df.drop(['expenses', 'sex', 'smoker'], axis=1)
y = df['expenses']

X_train_processed, X_test_processed, y_train, y_test = train_test_split(X, y, test_

region_cols = [col for col in X_train_processed.columns if col.startswith('region_'
for col in region_cols:
    X_train_processed[col] = X_train_processed[col].astype(int)
    X_test_processed[col] = X_test_processed[col].astype(int)

print("X_train_processed head:")
display(X_train_processed.head())
print("X_test_processed head:")
display(X_test_processed.head())
```

X_train_processed head:

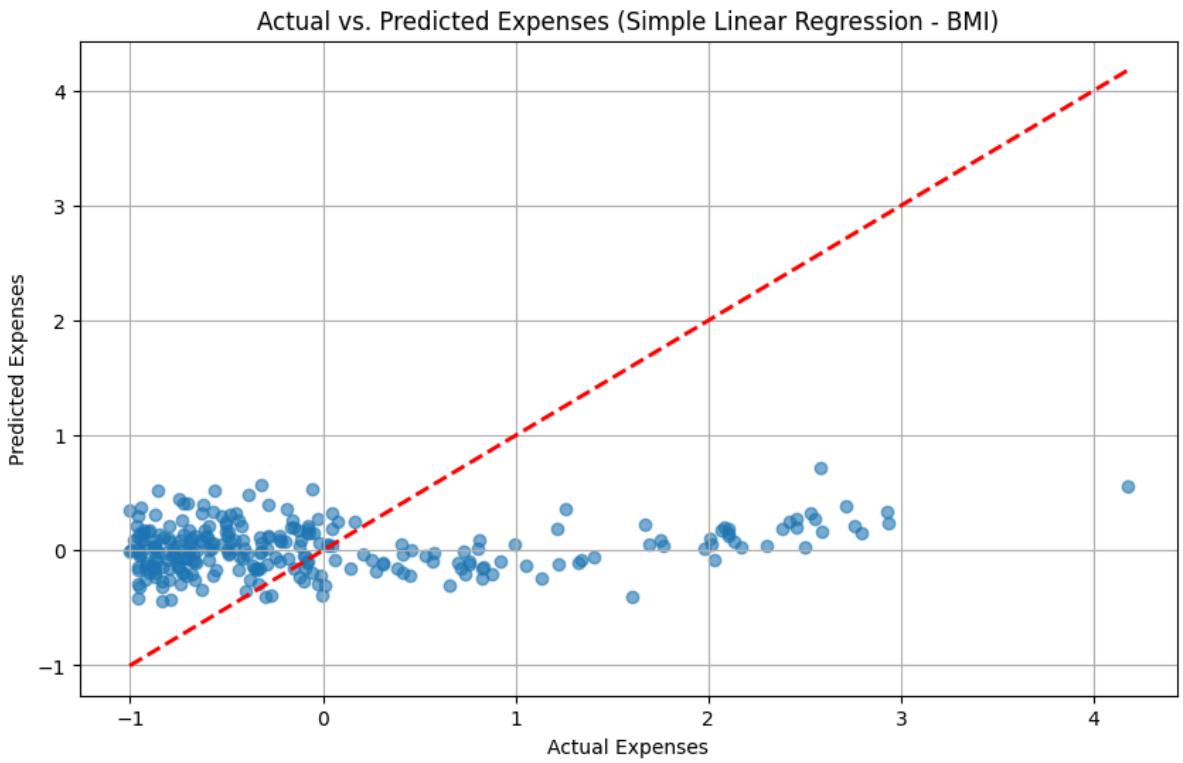
	age	bmi	children	sex_encoded	smoker_encoded	region_northeast	region_nor
560	0.483668	-1.749556	0.751079	0	0	0	
1285	0.554869	-1.044187	-0.908614	0	0	1	
1142	0.910875	-0.945763	-0.908614	0	0	0	
969	-0.014740	0.596205	3.240619	0	0	0	
486	1.053277	-1.503497	1.580926	0	0	0	

X_test_processed head:

	age	bmi	children	sex_encoded	smoker_encoded	region_northeast	region_nor
764	0.412467	-0.896552	0.751079	0	0	1	
887	-0.228344	-0.109163	-0.908614	0	0	0	
890	1.765289	-0.617685	-0.908614	0	1	0	
1293	0.483668	-0.814532	1.580926	1	0	0	
259	-1.438764	0.202511	-0.908614	1	1	0	

In [ ]:
```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, root

# 1. Instantiate a LinearRegression model
slr_bmi_model = LinearRegression()

# 2. Train the model using 'bmi' only
slr_bmi_model.fit(X_train_processed[['bmi']], y_train)

# 3. Make predictions on the test set
y_pred_slr_bmi = slr_bmi_model.predict(X_test_processed[['bmi']])

# 4. Calculate MAE
mae_slr_bmi = mean_absolute_error(y_test, y_pred_slr_bmi)
```

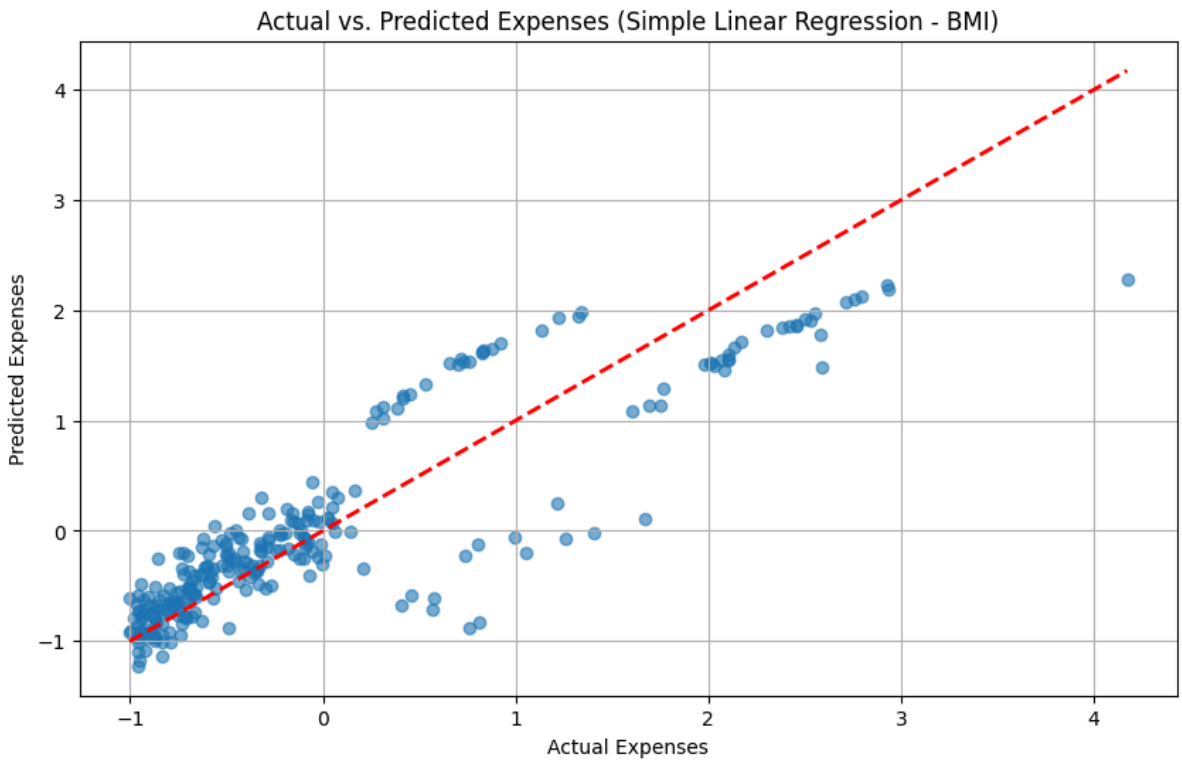
```
# 5. Calculate MSE
mse_slr_bmi = mean_squared_error(y_test, y_pred_slr_bmi)

# 6. Calculate RMSE
rmse_slr_bmi = root_mean_squared_error(y_test, y_pred_slr_bmi)

# 7. Calculate R² Score
r2_slr_bmi = r2_score(y_test, y_pred_slr_bmi)

# Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_slr_bmi, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Simple Linear Regression - BMI)')
plt.grid(True)
plt.show()
```



In [ ]:

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, root

# 1. Instantiate a MutliLinearRegression model
mr_model = LinearRegression()

# 2. Train the model using 'bmi' only
mr_model.fit(X_train_processed, y_train)

# 3. Make predictions on the test set
y_pred_mr = mr_model.predict(X_test_processed)

# 4. Calculate MAE
mae_mr = mean_absolute_error(y_test, y_pred_mr)

# 5. Calculate MSE
mse_mr = mean_squared_error(y_test, y_pred_mr)
```

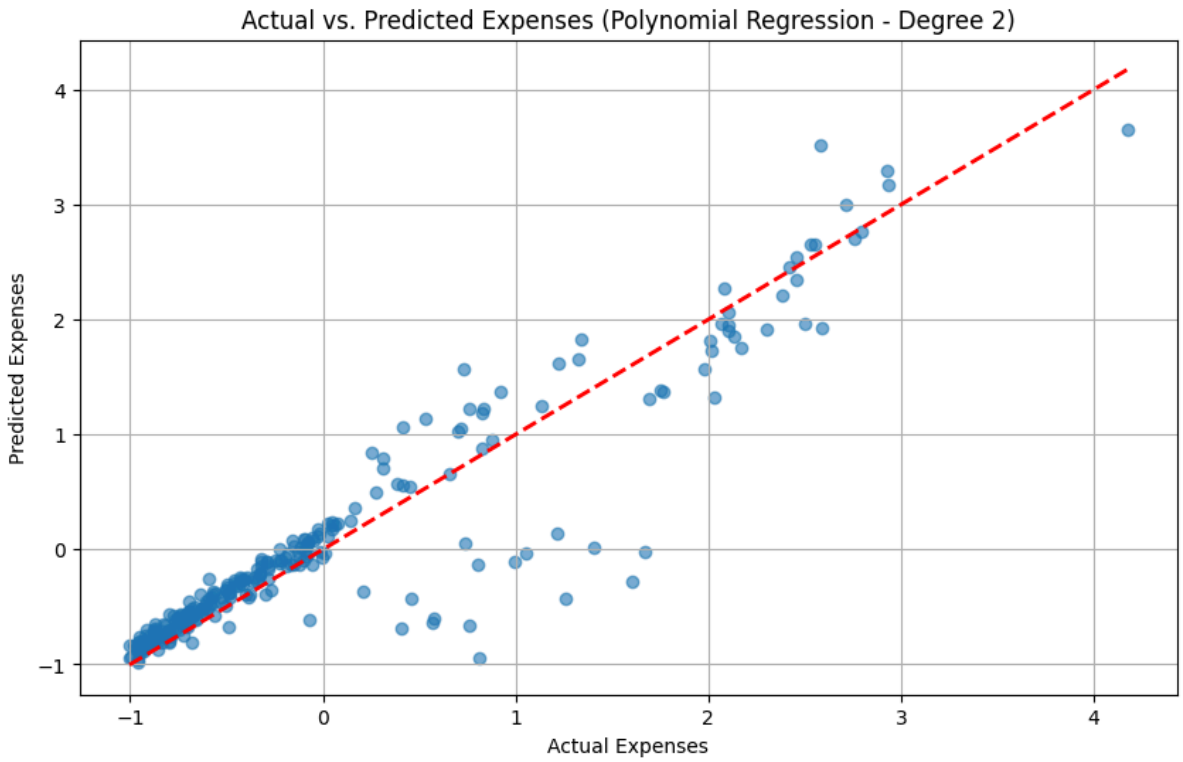
```
# 6. Calculate RMSE
rmse_mr = root_mean_squared_error(y_test, y_pred_mr)

# 7. Calculate R² Score
r2_mr = r2_score(y_test, y_pred_mr)

#  Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_mr, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Simple Linear Regression - BMI)')
plt.grid(True)
plt.show()
```



In [ ]:

```
# 1. Instantiate PolynomialFeatures with degree=2
poly = PolynomialFeatures(degree=2)

# 2. Transform X_train_processed and X_test_processed
X_train_poly2 = poly.fit_transform(X_train_processed)
X_test_poly2 = poly.transform(X_test_processed)

# 3. Instantiate a LinearRegression model
poly_reg_model2 = LinearRegression()

# 4. Train the model
poly_reg_model2.fit(X_train_poly2, y_train)

# 5. Make predictions on the test set
y_pred_poly2 = poly_reg_model2.predict(X_test_poly2)

# 6. Calculate evaluation metrics
mae_poly2 = mean_absolute_error(y_test, y_pred_poly2)
mse_poly2 = mean_squared_error(y_test, y_pred_poly2)
rmse_poly2 = root_mean_squared_error(y_test, y_pred_poly2)
r2_poly2 = r2_score(y_test, y_pred_poly2)
```

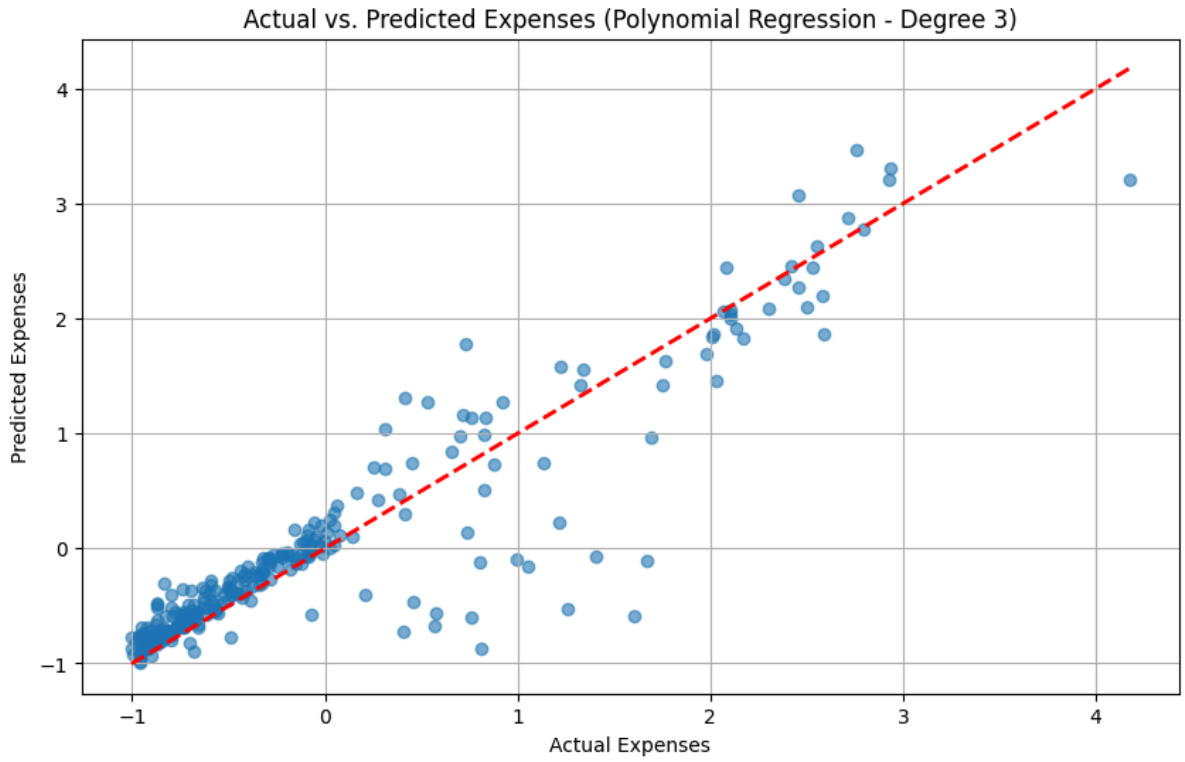
```
#  Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_poly2, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Polynomial Regression - Degree 2)')
plt.grid(True)
plt.show()
```



In [ ]:

```
# 1. Instantiate PolynomialFeatures with degree=3
poly3 = PolynomialFeatures(degree=3)

# 2. Transform X_train_processed and X_test_processed
X_train_poly3 = poly3.fit_transform(X_train_processed)
X_test_poly3 = poly3.transform(X_test_processed)

# 3. Instantiate a LinearRegression model
poly_reg_model3 = LinearRegression()

# 4. Train the model
poly_reg_model3.fit(X_train_poly3, y_train)

# 5. Make predictions on the test set
y_pred_poly3 = poly_reg_model3.predict(X_test_poly3)

# 6. Calculate evaluation metrics
mae_poly3 = mean_absolute_error(y_test, y_pred_poly3)
mse_poly3 = mean_squared_error(y_test, y_pred_poly3)
rmse_poly3 = root_mean_squared_error(y_test, y_pred_poly3)
r2_poly3 = r2_score(y_test, y_pred_poly3)

# Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_poly3, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Polynomial Regression - Degree 3)')
```

```
plt.grid(True)
plt.show()
```

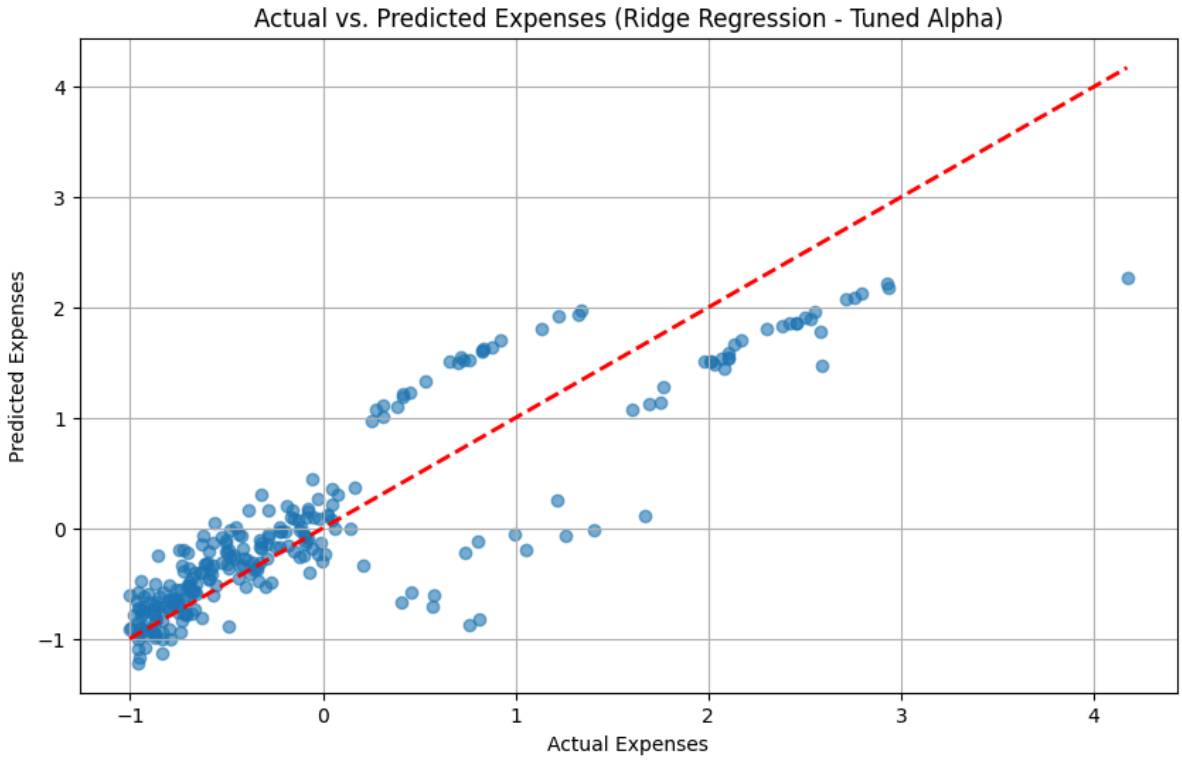


In [ ]:

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, root

# 1. Define a range of alpha values for tuning
alphas = [0.001, 0.01, 0.1, 1, 10, 100]
param_grid = {'alpha': alphas}

# 2. Instantiate a Ridge model
ridge = Ridge()

# 3. Instantiate GridSearchCV with the Ridge model and alpha parameter grid
grid_search = GridSearchCV(ridge, param_grid, cv=5, scoring='neg_mean_squared_error

# 4. Fit GridSearchCV to X_train_processed and y_train
grid_search.fit(X_train_processed, y_train)

# 5. Get the best Ridge model
best_ridge_model = grid_search.best_estimator_
print(f"Best alpha for Ridge Regression: {grid_search.best_params_['alpha']}")

# 6. Make predictions on X_test_processed using the best model
y_pred_ridge = best_ridge_model.predict(X_test_processed)

# 7. Calculate and print evaluation metrics
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
rmse_ridge = root_mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

# Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_ridge, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
```

```
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Ridge Regression - Tuned Alpha)')
plt.grid(True)
plt.show()
```

```
Best alpha for Ridge Regression: 1
```

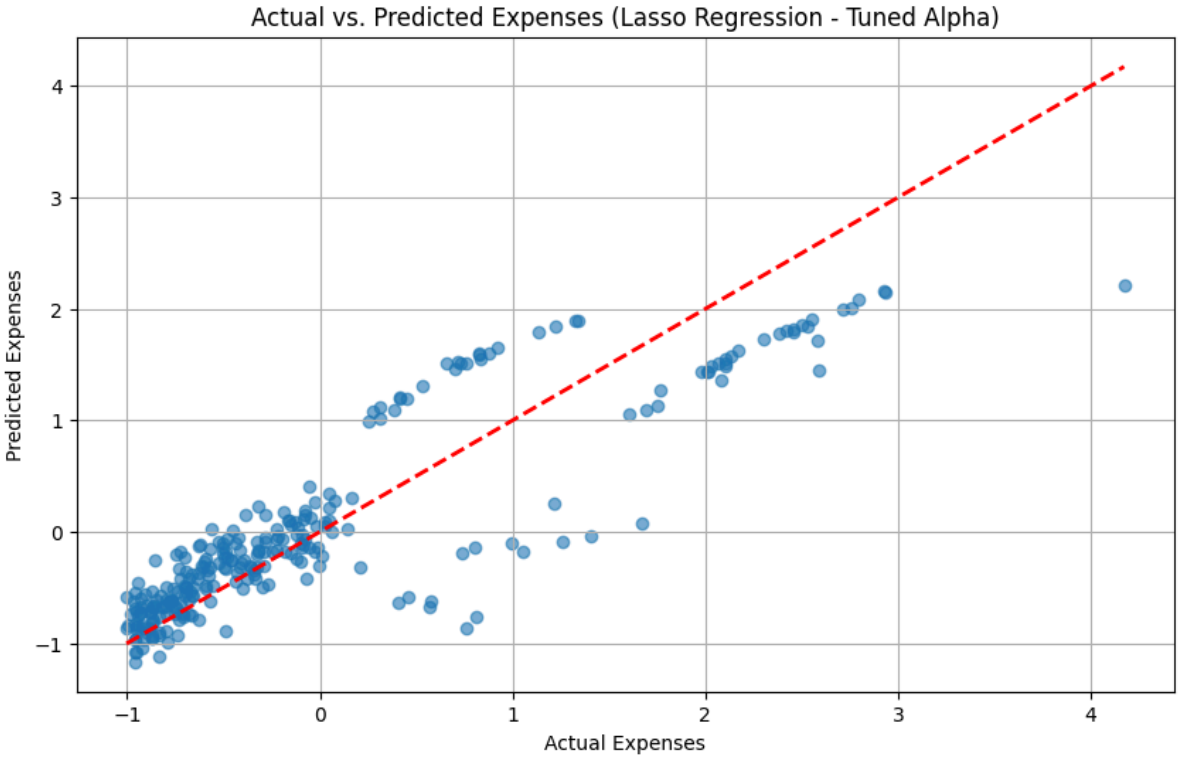


In [ ]:

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, root

# 1. Define a range of alpha values for tuning
alphas = [0.001, 0.01, 0.1, 1, 10, 100]
param_grid = {'alpha': alphas}

# 2. Instantiate a Lasso model
lasso = Lasso(random_state=42)

# 3. Instantiate GridSearchCV with the Lasso model and alpha parameter grid
grid_search = GridSearchCV(lasso, param_grid, cv=5, scoring='neg_mean_squared_error

# 4. Fit GridSearchCV to X_train_processed and y_train
grid_search.fit(X_train_processed, y_train)

# 5. Get the best Lasso model
best_lasso_model = grid_search.best_estimator_
print(f"Best alpha for Lasso Regression: {grid_search.best_params_['alpha']}")

# 6. Make predictions on X_test_processed using the best model
y_pred_lasso = best_lasso_model.predict(X_test_processed)

# 7. Calculate and print evaluation metrics
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = root_mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

# Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
```

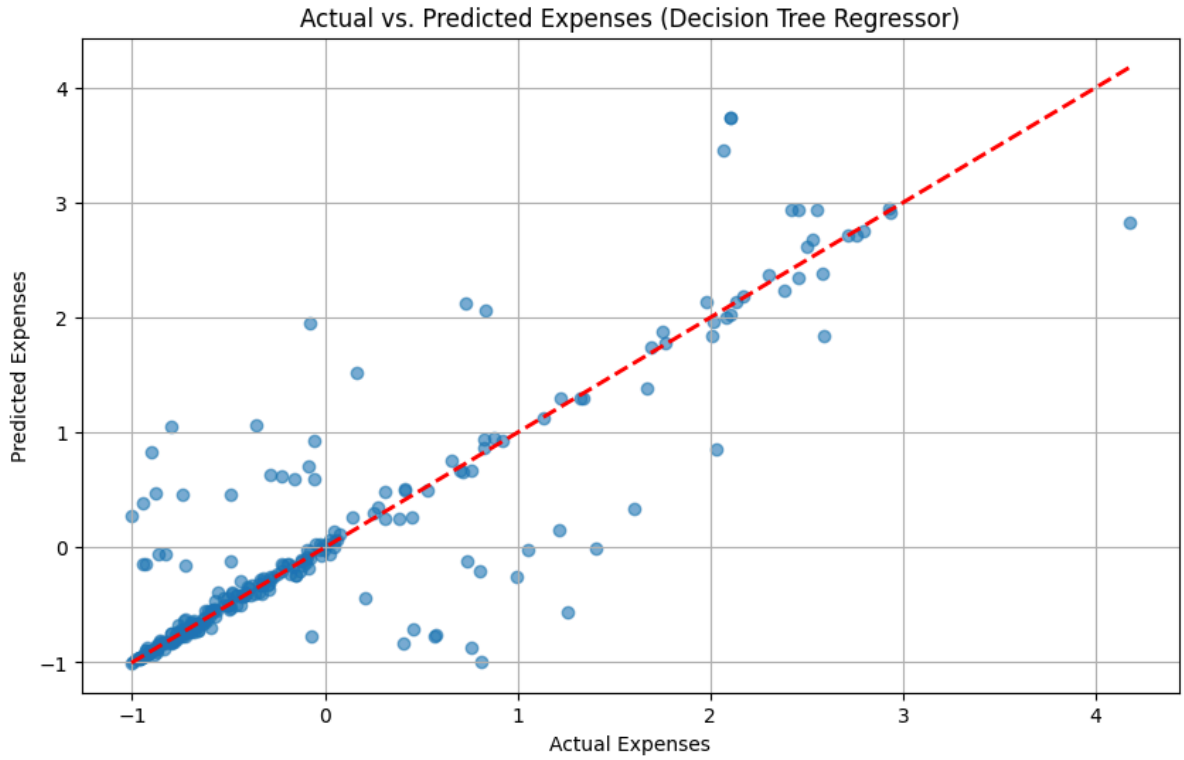
```
plt.scatter(y_test, y_pred_lasso, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Lasso Regression - Tuned Alpha)')
plt.grid(True)
plt.show()
```

```
Best alpha for Lasso Regression: 0.01
```



In [ ]:

```
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt

# 1. Instantiate a DecisionTreeRegressor model
dt_model = DecisionTreeRegressor(random_state=42)

# 2. Train the model using X_train_processed and y_train
dt_model.fit(X_train_processed, y_train)

# 3. Make predictions on the test set
y_pred_dt = dt_model.predict(X_test_processed)

# 4. Calculate evaluation metrics
mae_dt = mean_absolute_error(y_test, y_pred_dt)
mse_dt = mean_squared_error(y_test, y_pred_dt)
rmse_dt = root_mean_squared_error(y_test, y_pred_dt)
r2_dt = r2_score(y_test, y_pred_dt)

# Create a scatter plot of Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_dt, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Decision Tree Regressor)')
plt.grid(True)
plt.show()
```



In [ ]:

```
import pandas as pd

metrics_data = {
    'Model': [
        'Simple Linear Regression (BMI only)',
        'Multiple Linear Regression',
        'Polynomial Regression (Degree 2)',
        'Polynomial Regression (Degree 3)',
        'Ridge Regression',
        'Lasso Regression',
        'Decision Tree Regressor'
    ],
    'MAE': [
        mae_slr_bmi, mae_mr, mae_poly2, mae_poly3, mae_ridge, mae_lasso, mae_dt
    ],
    'MSE': [
        mse_slr_bmi, mse_mr, mse_poly2, mse_poly3, mse_ridge, mse_lasso, mse_dt
    ],
    'RMSE': [
        rmse_slr_bmi, rmse_mr, rmse_poly2, rmse_poly3, rmse_ridge, rmse_lasso, rmse
    ],
    'R² Score': [
        r2_slr_bmi, r2_mr, r2_poly2, r2_poly3, r2_ridge, r2_lasso, r2_dt
    ]
}

metrics_df = pd.DataFrame(metrics_data)
display(metrics_df)
```

	Model	MAE	MSE	RMSE	R^2 Score
0	Simple Linear Regression (BMI only)	0.345427	0.229285	0.478837	0.783573
1	Multiple Linear Regression	0.345427	0.229285	0.478837	0.783573
2	Polynomial Regression (Degree 2)	0.225516	0.141269	0.375857	0.866653
3	Polynomial Regression (Degree 3)	0.242647	0.160237	0.400296	0.848749
4	Ridge Regression	0.346397	0.229601	0.479167	0.783274
5	Lasso Regression	0.351595	0.234414	0.484163	0.778732
6	Decision Tree Regressor	0.240019	0.263986	0.513795	0.750818

Exercise 3: Artificial Neural Network (ANN) for Regression

Step 1: ANN Model Design

Design a Feed-Forward Neural Network for regression with:

- Input layer: number of neurons = number of features
- Hidden layer(s): 1 or 2 hidden layers
- Activation function:
- Hidden layer: ReLU
- Output layer: Linear activation
- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam (default parameters)

In [ ]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

n_features = X_train_processed.shape[1]
print(f"Number of features {n_features}")

model = Sequential()
model.add(Dense(16, activation='relu', input_shape=(n_features,)))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='linear'))
model.compile(optimizer='adam', loss='mse')
```

```
Number of features 9
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWar
ning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Seq
uential models, prefer using an `Input(shape)` object as the first layer in the mo
del instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Step 2: Model Training

- Compile the ANN model
- Train using training data

- Use validation split (e.g., 10–20%)
- Train for suitable epochs (e.g., 100)

In [ ]:
```
# Train the model
history = model.fit(
    X_train_processed, y_train,
    validation_split=0.2,  # 20% of training data for validation
    epochs=100,
    batch_size=32,
    verbose=0 # Set verbose to 1 or 2 for more output during training
)

print("ANN model trained successfully.")
```

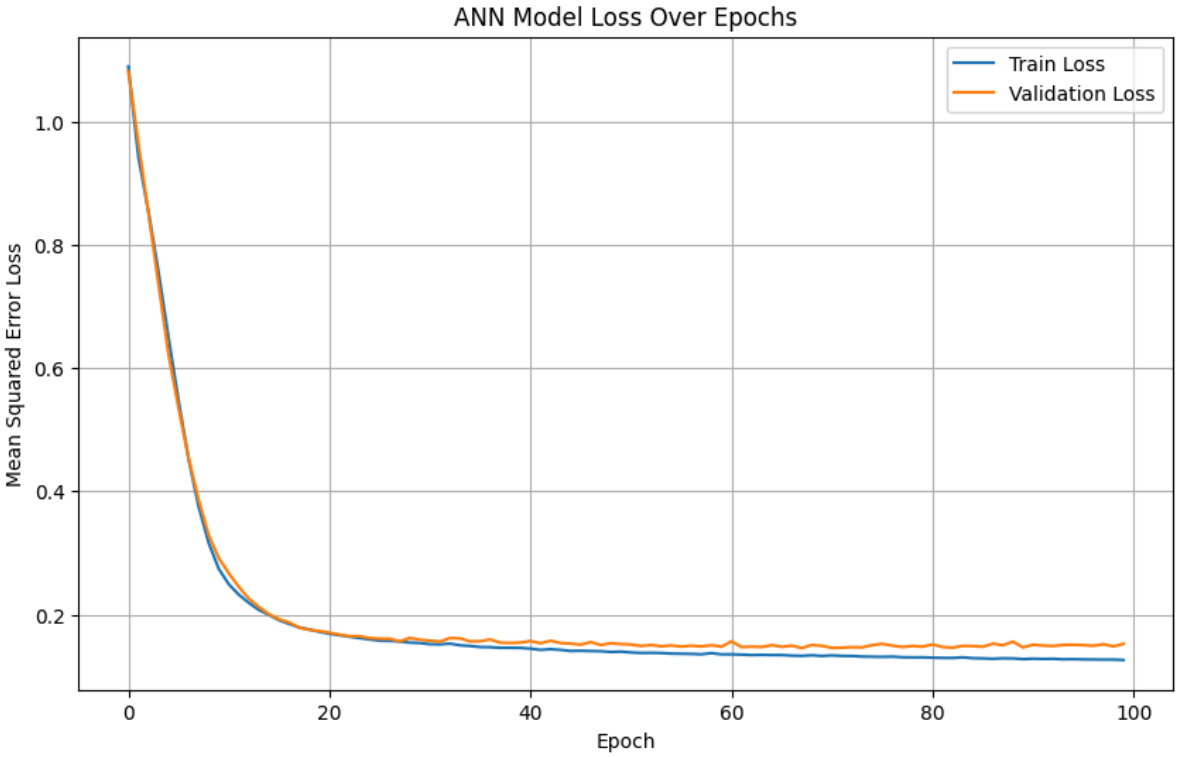
```
ANN model trained successfully.
```

Step 3: Model Evaluation

- Predict on test data
- Compute:
- MAE
- MSE
- RMSE
- R^2 Score
- Plot:
- Training vs Validation loss curve
- Actual vs Predicted scatter plot

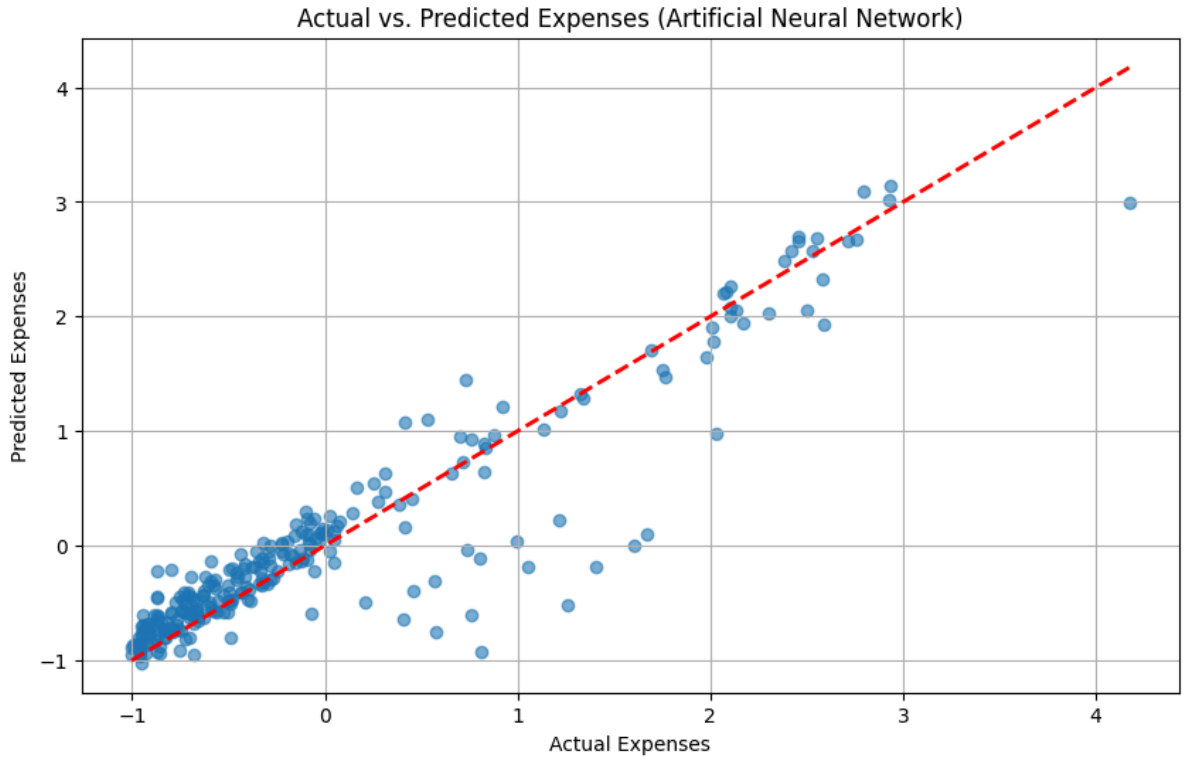
In [ ]:
```
import matplotlib.pyplot as plt

# Make predictions on the test data
y_pred_ann = model.predict(X_test_processed).flatten()

# Calculate evaluation metrics
mae_ann = mean_absolute_error(y_test, y_pred_ann)
mse_ann = mean_squared_error(y_test, y_pred_ann)
rmse_ann = root_mean_squared_error(y_test, y_pred_ann)
r2_ann = r2_score(y_test, y_pred_ann)

print("\nArtificial Neural Network Metrics:")
print(f"  MAE: {mae_ann:.4f}")
print(f"  MSE: {mse_ann:.4f}")
print(f"  RMSE: {rmse_ann:.4f}")
print(f"  R² Score: {r2_ann:.4f}")

# Plot Training vs Validation Loss
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('ANN Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error Loss')
plt.legend()
plt.grid(True)
plt.show()
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step

Artificial Neural Network Metrics:
  MAE: 0.2315
  MSE: 0.1441
  RMSE: 0.3796
  R² Score: 0.8640
```



In [ ]:

```
# Plot Actual vs Predicted values
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_ann, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) #
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs. Predicted Expenses (Artificial Neural Network)')
plt.grid(True)
plt.show()
```



Knowledge Check

1. Why is linear activation used in the output layer of ANN for regression?

In regression tasks, the goal is to predict a continuous numerical value (e.g., expenses, price, temperature). A linear activation function (which essentially means no activation function, or an identity function) allows the output neuron to produce any real number. Other activation functions like sigmoid or ReLU would squash the output into a specific range (e.g., between 0 and 1 for sigmoid, or non-negative for ReLU), which is suitable for classification or specific types of regression (like predicting probabilities), but not for general continuous value prediction.

2. How does ANN differ from Polynomial Regression?

- Polynomial Regression: This is a form of linear regression where the relationship between the independent variable(s) and the dependent variable is modeled as an nth-degree polynomial. It transforms the original features into polynomial terms (e.g., x, x^2, x^3) and then fits a linear model to these transformed features. It's still fundamentally a linear model in terms of its parameters.
- Artificial Neural Network (ANN): ANNs are highly flexible, non-linear models composed of multiple layers of interconnected neurons. Each neuron applies a non-linear activation function (like ReLU in hidden layers) to a weighted sum of its inputs. This layered structure and non-linear activations allow ANNs to learn complex, hierarchical representations of the data and model highly intricate, non-linear relationships without

explicitly defining polynomial terms. ANNs are generally capable of learning much more abstract and complex patterns than polynomial regression.

3. What role does optimizer play in ANN training?

The optimizer is the algorithm used to adjust the internal parameters (weights and biases) of the neural network during the training process. Its primary role is to minimize the model's loss function (e.g., Mean Squared Error for regression) by iteratively updating these parameters based on the gradients of the loss with respect to the parameters. Optimizers like Adam (which you used), SGD, RMSprop, or Adagrad determine the learning rate and direction of these updates, guiding the network towards a state where it makes more accurate predictions.

4. Why does ANN require feature scaling?

ANNs, especially those trained with gradient-based optimizers, are very sensitive to the scale of input features. If features have vastly different ranges, the gradients for features with larger values will tend to be larger, leading to disproportionate weight updates. This can cause:

- Slower Convergence: The optimizer might struggle to find the optimal path to the minimum of the loss function.
- Oscillations: The training process can become unstable, oscillating wildly instead of smoothly converging.
- Difficulty in Weight Initialization: Initial weights might lead to very large or very small activations, causing issues like vanishing or exploding gradients.
  Feature scaling (normalization or standardization) brings all features to a similar scale, ensuring that each feature contributes equally to the learning process and helps the optimizer converge more efficiently and stably.

5. Compare overfitting behavior of Decision Tree and ANN.

- Decision Trees: Are notoriously prone to overfitting, especially when allowed to grow to full depth without any regularization. A deep decision tree can learn the training data, including noise, almost perfectly, leading to excellent training performance but poor generalization to unseen data. Techniques like pruning, setting max_depth, or min_samples_leaf are used to mitigate this.
- Artificial Neural Networks: Can also overfit, particularly deep or wide networks with many parameters relative to the amount of training data. A powerful ANN can memorize the training examples rather than learning general patterns. Overfitting in ANNs is typically addressed using various regularization techniques such as L1/L2 regularization, dropout (randomly dropping neurons during training), early stopping (halting training when validation performance starts to degrade), and providing a sufficiently large and diverse dataset.

6. When would ML regression models be preferred over ANN?

- Interpretability is Key: For applications where understanding why a prediction is made (e.g., identifying the most important features) is as crucial as the prediction itself, simpler models like Linear Regression, Ridge/Lasso, or even simpler Decision Trees are often preferred due to their inherent interpretability.
- Limited Data: ANNs typically require a large amount of data to generalize well. With small datasets, simpler ML models might perform better and be less prone to overfitting.
- Computational Resources: Training deep ANNs can be computationally intensive and require significant hardware (GPUs). Simpler ML models are much faster to train on standard CPUs.
- Simpler Relationships: If the underlying relationship between features and the target variable is relatively straightforward and can be well-captured by linear or simple non-linear functions, then a less complex ML model might offer comparable performance with less overhead.
- Establishing Baselines: Simpler ML models are often used as a baseline to evaluate whether the complexity of an ANN is truly justified by a significant performance improvement.

7. How does batch size affect ANN training?

- Batch size refers to the number of training examples utilized in one iteration of training before the model's internal parameters (weights and biases) are updated. It significantly impacts training dynamics: Large Batch Size:
- Pros: Provides a more accurate estimate of the gradient, leading to more stable updates. Often leads to faster training per epoch (fewer updates). Better utilization of computational resources like GPUs.
- Cons: Can converge to sharp minima, which might generalize poorly to unseen data. Requires more memory. Updates are less frequent, potentially slowing down overall convergence in terms of wall-clock time if the dataset is very large. Small Batch Size (e.g., 1 - Stochastic Gradient Descent): Pros: Introduces more noise into the gradient updates, which can help the model escape shallow local minima and find flatter, more generalizable minima. Requires less memory. Leads to more frequent updates. Cons: Very noisy gradient estimates can lead to unstable training and oscillations. Slower training per epoch (many updates). Less efficient use of computational resources.
- Mini-batch (common compromise): An intermediate batch size (e.g., 32, 64, 128) is often chosen as a trade-off, balancing the stability of large batches with the generalization benefits and memory efficiency of small batches. The optimal batch size is often found through experimentation.